ASSIGNMENT NO: 4

AIM : Implement the deadlock-free solution to Dining Philosophers problem to illustrate the problem of deadlock and/or starvation that can occur when many synchronized threads are competing for limited resources.

Objective : To implement a **deadlock-free solution to the Dining Philosophers Problem** in order to:

- Understand the concepts of **deadlock** and **starvation** in concurrent systems.
- Demonstrate how multiple synchronized threads compete for limited shared resources.
- Analyze the conditions that lead to deadlock and starvation.
- Apply synchronization techniques (such as semaphores, mutexes, or monitors) to prevent deadlock.
- Ensure fair resource allocation so that every philosopher eventually gets access to the required resources without indefinite waiting.
- Study process synchronization and efficient resource-sharing mechanisms used in operating systems.

Theory:

The **Dining Philosophers Problem** is a classic synchronization problem in operating systems, introduced by **Edsger W. Dijkstra** to illustrate the challenges of resource allocation and process synchronization. The problem demonstrates how multiple processes (or threads) compete for a limited number of shared resources, which can lead to **deadlock** or **starvation** if not managed properly.

In the problem, **five philosophers** are seated around a circular table. A **fork (or chopstick)** is placed between each pair of adjacent philosophers, resulting in five forks. Each philosopher alternates between **thinking** and **eating**. To eat, a philosopher must acquire **both the left and right forks**. Since each fork is shared between two neighboring philosophers, only one philosopher can hold a fork at a time.

If every philosopher picks up the left fork simultaneously and waits for the right fork, each philosopher holds one fork while waiting for another. This creates a **deadlock**, where no philosopher can proceed because each is waiting for a resource held by another philosopher. Another issue is **starvation**, where a philosopher may wait indefinitely if other philosophers continuously acquire the required forks before them.

To eliminate these problems, a **deadlock-free synchronization strategy** is implemented. Common techniques include:

- Using **mutex locks** or **semaphores** to control access to forks.
- Allowing only **N−1 philosophers** to attempt eating simultaneously.
- Requiring philosophers to pick up forks in a fixed order (resource hierarchy).
- Using a **monitor** with condition variables to ensure safe and fair resource allocation.

These techniques satisfy the synchronization requirements by ensuring that:

- Mutual exclusion is maintained for shared resources.
- Circular waiting is prevented.
- Deadlock does not occur.
- Every philosopher eventually gets an opportunity to eat, reducing or eliminating starvation.

The Dining Philosophers Problem is widely used to study **process synchronization**, **thread coordination**, **resource allocation**, and **concurrency control** in operating systems. It demonstrates the practical use of synchronization primitives such as **semaphores**, **mutexes**, **condition variables**, and **monitors** for designing efficient and reliable concurrent applications.

Applications

- Operating system process synchronization
- Multithreaded programming
- Database transaction management
- Resource allocation in distributed systems
- Network and cloud resource scheduling

Outcome

By implementing a deadlock-free solution to the Dining Philosophers Problem, students learn how synchronization mechanisms prevent deadlock and starvation, ensure safe access to shared resources, and improve the reliability and efficiency of concurrent systems.

Program 4:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_mutex_t fork_mutex[N];
sem_t room;

void *philosopher(void *arg)
{
   int id = *(int *)arg;

   int left_fork = id;
   int right_fork = (id + 1) % N;

   printf("Philosopher %d is THINKING.\n", id + 1);
   sleep(1);

   /* Allow only N-1 philosophers to compete for forks */
   sem_wait(&room);

   /* Pick up left fork */
   pthread_mutex_lock(&fork_mutex[left_fork]);
   printf("Philosopher %d picked up left fork.\n", id + 1);

   /* Pick up right fork */
   pthread_mutex_lock(&fork_mutex[right_fork]);
   printf("Philosopher %d picked up right fork.\n", id + 1);

   /* Eat only once */
   printf("Philosopher %d is EATING.\n", id + 1);
   sleep(2);

   /* Release both forks */
   pthread_mutex_unlock(&fork_mutex[right_fork]);
   pthread_mutex_unlock(&fork_mutex[left_fork]);

   printf("Philosopher %d finished eating.\n", id + 1);

   /* Allow another philosopher to enter */
   sem_post(&room);

   return NULL;
}

int main()
```

```
{
    pthread_t philosophers[N];
    int id[N];

    /* Initialize fork mutexes */
    for (int i = 0; i < N; i++)
    {
        pthread_mutex_init(&fork_mutex[i], NULL);
    }

    /*
     * Only N-1 philosophers can compete for
     * forks at the same time.
     */
    sem_init(&room, 0, N - 1);

    /* Create philosopher threads */
    for (int i = 0; i < N; i++)
    {
        id[i] = i;

        pthread_create(&philosophers[i],
                       NULL,
                       philosopher,
                       &id[i]);
    }

    /* Wait for all philosophers */
    for (int i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }

    /* Destroy mutexes */
    for (int i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&fork_mutex[i]);
    }

    sem_destroy(&room);

    printf("\nAll philosophers have eaten exactly once.\n");
    printf("Deadlock-free execution completed successfully.\n");

    return 0;
}
```

CONCLUSION: Thus, deadlock-free solution to Dining Philosophers problem is implemented.